AMRITA VISHWA VIDYAPEETHAM

Name: Ganath Avinash G.R

CSE – B

CH.SC.U4CSE24118

Week –6 (Quick Select)

1. Quick Select

Code:

```
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function (last element as pivot)
int partition(int arr[], int low, int high) {
    int pivot = arr[high];   // last element as pivot
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1; // pivot index
}

// Quick Select function
int quickSelect(int arr[], int low, int high, int k) {
    if (low <= high) {
        int pivotIndex = partition(arr, low, high);
```

```
        if (pivotIndex == k)
            return arr[pivotIndex];
        else if (pivotIndex > k)
            return quickSelect(arr, low, pivotIndex - 1, k);
        else
            return quickSelect(arr, pivotIndex + 1, high, k);
    }
    return -1; // error case
}

// Main function
int main() {
    int arr[] = {12, 3, 5, 7, 19, 26, 4};
    int n = sizeof(arr) / sizeof(arr[0]);
    int k = 3; // k-th smallest (1-based)

    int result = quickSelect(arr, 0, n - 1, k - 1);

    printf("The %d-th smallest element is %d\n", k, result);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Ganath Avinash\OneDrive\ドキュメント\Back-end\DAA> cd
?) { .\Quickselect }
The 3-th smallest element is 5
PS C:\Users\Ganath Avinash\OneDrive\ドキュメント\Back-end\DAA>
```

Complexity:

Space Complexity:O(1)
Time Complexity:O(n^2)

Reasoning:

Quick Select is faster than Quick Sort for k-th element
Because it discards half of the array each step